

R Syntax and Code Cheatsheet

Statistical Modeling Support Group

If the response is	Start with	Main R function	Typical evaluation
Continuous	Linear regression	lm()	RMSE, residuals, R^2
Binary	Logistic regression	glm(..., binomial)	Confusion matrix, accuracy
Unordered classes	LDA/QDA/NB or multinomial	lda(), qda(), naiveBayes()	Confusion matrix, accuracy
Ordered classes	Ordinal logistic	polr()	Class predictions, coefficients
Count	Poisson regression	glm(..., poisson)	Deviance, residuals, predictions
Positive, right-skewed	Gamma regression	glm(..., Gamma)	Deviance, residuals, predictions

Remember. Replace placeholder names such as `dat`, `y`, `x1` and `group` with the actual data-frame and variable names. Run code in order; later objects depend on earlier objects.

1 R Essentials

1.1 Assignment, comments and functions

```
# Anything after # is a comment
x <- 10          # assign a value
y <- c(4, 7, 9, 12) # create a vector
mean(y)         # call a function
sqrt(x)         # named function, one argument
round(mean(y), digits = 2)

# Print or inspect an object
x
print(x)
class(y)
length(y)
```

General function pattern:

```
result <- function(data, argument = value).
```

1.2 Useful object constructors

```
c(2, 4, 6)          # vector
1:5                 # 1, 2, 3, 4, 5
seq(0, 1, by = 0.2) # regular sequence
rep(0, times = 5)   # repeat a value

df <- data.frame(
  score = c(70, 82, 65),
  group = c("A", "B", "A")
)
```

1.3 Operators

Operator	Meaning
<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>^</code>	Arithmetic
<code>==</code> <code>!=</code>	Equal to; not equal to
<code><</code> <code><=</code> <code>></code> <code>>=</code>	Comparisons
<code>&</code> <code> </code>	Element-wise AND; OR
<code>!</code>	NOT
<code>%in%</code>	Value occurs in a set
<code>~</code>	Separates response and predictors
<code>\$</code>	Selects a named column
<code>[]</code>	Selects rows, columns or elements

```
df$score >= 70
df$group == "A"
df$group %in% c("A", "B")
```

2 Import and Inspect Data

2.1 Import

```
dat <- read.csv("data.csv")

dat <- read.table("data.txt",
                 header = TRUE)

# Built-in data
data("iris")
data("Boston", package = "MASS")
```

Use forward slashes in file paths:

```
dat <- read.csv("C:/Users/name/data.csv")
```

2.2 Inspect before modelling

```
head(dat)          # first 6 rows
tail(dat)          # final 6 rows
dim(dat)           # rows and columns
nrow(dat); ncol(dat)
names(dat)         # column names
str(dat)           # types and structure
summary(dat)       # variable summaries
class(dat$y)       # one variable's type
unique(dat$group)  # distinct values
table(dat$group)   # category counts
```

Remember. Never fit a model before checking the response type, variable names, missing values and category coding.

3 Select, Filter and Clean

3.1 Rows, columns and conditions

```
dat$y              # one column as vector
dat[, "y"]         # one named column
dat[, c("y", "x1")] # selected columns
dat[1:10, ]        # first 10 rows
dat[dat$group == "A", ] # filtered rows

subset(dat, group == "A",
       select = c(y, x1, group))

# Remove one column
dat2 <- dat[, names(dat) != "id"]
```

3.2 Missing values

```
is.na(dat)         # TRUE/FALSE matrix
```

```
sum(is.na(dat))      # total missing cells
colSums(is.na(dat))  # missing by column
complete.cases(dat)  # complete-row indicator
dat_clean <- na.omit(dat)
```

Important: `na.omit()` removes an entire row if any selected variable is missing. Always check the remaining sample size.

3.3 Factors and reference categories

```
dat$group <- factor(dat$group)
levels(dat$group)
table(dat$group)
contrasts(dat$group)

# Select a reference category
dat$group <- relevel(dat$group, ref = "A")

# Ordered categories
dat$satisfaction <- ordered(
  dat$satisfaction,
  levels = c("Low", "Medium", "High")
)
```

Read the output. In a model with an intercept, R creates $k - 1$ dummy variables for a factor with k levels. Every dummy coefficient compares its category with the reference category.

4 Explore the Data

4.1 Numerical summaries

```
mean(dat$y, na.rm = TRUE)
median(dat$y, na.rm = TRUE)
sd(dat$y, na.rm = TRUE)
var(dat$y, na.rm = TRUE)
quantile(dat$y, na.rm = TRUE)

# Summary by group
aggregate(y ~ group, data = dat, mean)
tapply(dat$y, dat$group, mean)
```

4.2 Basic plots

```
hist(dat$y)
boxplot(y ~ group, data = dat)
plot(dat$x1, dat$y, pch = 16)
barplot(table(dat$group))

# Add a fitted line to a scatterplot
abline(lm(y ~ x1, data = dat), lwd = 2)
```

5 Hypothesis-Test Templates

5.1 One- and two-sample means

```
# H0: population mean = 15
t.test(dat$y, mu = 15)

# Independent samples, Welch default
t.test(y ~ group, data = dat)

# Equal-variance version
t.test(y ~ group, data = dat,
      var.equal = TRUE)

# Paired measurements
t.test(before, after, paired = TRUE)

# Known population SD
BSDA::z.test(dat$y, mu = 100,
             sigma.x = 15)
```

5.2 Variances and proportions

```
# H0: two population variances are equal
var.test(group1, group2)

# One proportion: 60 successes out of 100
prop.test(x = 60, n = 100, p = 0.50)

# Compare two proportions
prop.test(x = c(60, 80),
          n = c(100, 200))
```

5.3 ANOVA and post-hoc comparison

```
fit_aov <- aov(score ~ technique, data = dat)
summary(fit_aov)

# Equality of group variances
car::leveneTest(score ~ technique,
                 data = dat)

# Use after a significant overall ANOVA
agricolae::LSD.test(fit_aov, "technique")
```

5.4 Normality and correlation

```
shapiro.test(dat$y)

cor(dat$x1, dat$y,
    use = "complete.obs")
cor.test(dat$x1, dat$y)
```

Read the output. For a test at significance level $\alpha = 0.05$: if $p < 0.05$, reject H_0 ; otherwise, fail to reject H_0 . Report the practical conclusion, not only the p -value.

6 R Formula Syntax

Formula	Meaning
$y \sim x1$	Predict y using $x1$
$y \sim x1 + x2$	Two predictors
$y \sim .$	Use every other column
$y \sim x1 * \text{group}$	Main effects plus interaction
$y \sim x1:\text{group}$	Interaction only
$y \sim x1 - 1$	Remove the intercept

```
# Always identify the data source
fit <- lm(y ~ x1 + x2 + group, data = dat)
```

Remember. The left side of $\sim$ is the response. The right side lists predictors. The `data =` argument tells R where to find those variables.

7 Train–Test Split

7.1 Simple reproducible split

```
set.seed(42)
id <- sample(seq_len(nrow(dat)),
             size = round(0.8 * nrow(dat)))
train <- dat[id, ]
test  <- dat[-id, ]
```

7.2 Stratified split for classification

```
set.seed(42)
id <- caret::createDataPartition(
  dat$class, p = 0.8, list = FALSE
)
train <- dat[id, ]
test  <- dat[-id, ]

# Confirm class balance
prop.table(table(train$class))
prop.table(table(test$class))
```

Remember. Fit and tune models using training data. Use the test data only for final comparison. Give all competing models the same split.

8 Linear Regression

8.1 Fit, inspect and predict

```
fit_lm <- lm(y ~ x1 + x2 + group,
            data = train)

summary(fit_lm)
coef(fit_lm)
confint(fit_lm, level = 0.95)
fitted(fit_lm)
residuals(fit_lm)

pred_lm <- predict(fit_lm,
                  newdata = test)
```

8.2 Calculate MSE and RMSE

```
actual <- test$y
mse_lm <- mean((actual - pred_lm)^2)
rmse_lm <- sqrt(mse_lm)
```

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

Read the output. In `summary(fit_lm)`, use `Estimate` for coefficient direction and size, `Pr(>|t|)` for individual tests, adjusted R^2 for complexity-adjusted fit, and the final F -test for overall regression significance.

9 Linear-Model Diagnostics

```
# Four standard diagnostic plots
par(mfrow = c(2, 2))
plot(fit_lm)
par(mfrow = c(1, 1))

# Residuals versus fitted values
plot(fitted(fit_lm), residuals(fit_lm))
abline(h = 0, lty = 2)

# Residual normality
qqnorm(residuals(fit_lm))
qqline(residuals(fit_lm))
shapiro.test(residuals(fit_lm))

# Independence and multicollinearity
lmtest::dwtest(fit_lm)
car::vif(fit_lm)
```

Finding	Possible meaning
Curved residual pattern	Nonlinear relationship remains
Funnel shape	Nonconstant residual variance
Q–Q departures	Residual non-normality
DW below 2	Possible positive autocorrelation
Large VIF	Unstable coefficients from collinearity

10 Subset and Stepwise Selection

```
library(leaps)

best <- regsubsets(y ~ ., data = train,
                  nvmax = ncol(train) - 1)
best_sum <- summary(best)

cbind(RSS = best_sum$rsr,
      R2 = best_sum$rsr,
      Cp = best_sum$cp,
      BIC = best_sum$bic,
      AdjR2 = best_sum$adjr2)

which.min(best_sum$cp)
which.min(best_sum$bic)
which.max(best_sum$adjr2)

coef(best, id = which.min(best_sum$bic))
```

```
forward <- regsubsets(
  y ~ ., data = train,
  nvmax = ncol(train) - 1,
  method = "forward"
)

backward <- regsubsets(
  y ~ ., data = train,
  nvmax = ncol(train) - 1,
  method = "backward"
)
```

Preferred direction: minimise C_p or BIC; maximise adjusted R^2 . Confirm the final choice using validation or test performance.

11 Ridge and LASSO

11.1 Prepare matrices

```
library(glmnet)

# model.matrix creates dummy variables
x_train <- model.matrix(y ~ ., train)[, -1]
y_train <- train$y
x_test <- model.matrix(y ~ ., test)[, -1]
y_test <- test$y
```

11.2 Ridge: alpha = 0

```
set.seed(42)
cv_ridge <- cv.glmnet(
  x_train, y_train, alpha = 0
)
lambda_ridge <- cv_ridge$lambda.min

pred_ridge <- predict(
  cv_ridge, newx = x_test,
  s = "lambda.min"
)
rmse_ridge <- sqrt(mean(
  (y_test - as.vector(pred_ridge))^2
))
```

11.3 LASSO: alpha = 1

```
set.seed(42)
cv_lasso <- cv.glmnet(
  x_train, y_train, alpha = 1
)
lambda_lasso <- cv_lasso$lambda.min

pred_lasso <- predict(
  cv_lasso, newx = x_test,
  s = "lambda.min"
)
rmse_lasso <- sqrt(mean(
  (y_test - as.vector(pred_lasso))^2
))

lasso_coef <- coef(cv_lasso,
  s = "lambda.min")
selected <- rownames(lasso_coef)[
  as.vector(lasso_coef) != 0
]
selected <- setdiff(selected, "(Intercept)")
```

Value	Meaning
lambda.min	Lowest cross-validation error
lambda.1se	Simpler choice within one SE of minimum
alpha = 0	Ridge; shrinks all slopes
alpha = 1	LASSO; may set slopes to zero

Remember. Do not choose a model from training fit alone. Compare MLR, ridge and LASSO using the same test observations and the same error measure.

12 Binary Logistic Regression

12.1 Fit and interpret

```
fit_logit <- glm(
  outcome ~ x1 + x2 + group,
  data = train,
  family = binomial(link = "logit")
)

summary(fit_logit)
coef(fit_logit)          # log-odds coefficients
exp(coef(fit_logit))     # odds ratios
exp(confint(fit_logit))  # OR confidence intervals
```

$$\log\left(\frac{\pi}{1-\pi}\right) = X\beta, \quad \pi = \frac{e^{X\beta}}{1 + e^{X\beta}}.$$

12.2 Predict probabilities and classes

```
prob <- predict(fit_logit,
  newdata = test,
  type = "response")

pred_class <- ifelse(prob >= 0.50, 1, 0)
table(Actual = test$outcome,
  Predicted = pred_class)
mean(pred_class == test$outcome)
```

12.3 Compare models and assess fit

```
fit_null <- glm(outcome ~ 1,
  data = train, family = binomial)

anova(fit_null, fit_logit,
  test = "Chisq")
AIC(fit_null, fit_logit)

# Hosmer--Lemeshow test
ResourceSelection::hoslem.test(
  train$outcome, fitted(fit_logit), g = 10
)
```

Read the output. $e^{\hat{\beta}_j} > 1$ means the odds increase as X_j increases; a value below 1 means the odds decrease, holding other predictors constant. The classification threshold is a decision rule, not a fixed property of logistic regression.

13 LDA, QDA and Naive Bayes

13.1 Fit all models on the same training set

```
library(MASS)
library(e1071)

fit_lda <- lda(class ~ ., data = train)
fit_qda <- qda(class ~ ., data = train)
fit_nb <- naiveBayes(class ~ ., data = train)

pred_lda <- predict(fit_lda,
  newdata = test)$class
pred_qda <- predict(fit_qda,
  newdata = test)$class
pred_nb <- predict(fit_nb,
  newdata = test)
```

13.2 Evaluate fairly

```
cm_lda <- table(Actual = test$class,
  Predicted = pred_lda)
cm_qda <- table(Actual = test$class,
  Predicted = pred_qda)
cm_nb <- table(Actual = test$class,
  Predicted = pred_nb)

acc_lda <- mean(pred_lda == test$class)
acc_qda <- mean(pred_qda == test$class)
acc_nb <- mean(pred_nb == test$class)

c(LDA = acc_lda, QDA = acc_qda,
  NaiveBayes = acc_nb)
```

Method	Main modelling condition
LDA	Classes share one covariance matrix
QDA	Each class has its own covariance matrix
Naive Bayes	Predictors are conditionally independent within a class

Remember. Do not fit and evaluate Naive Bayes on the same observations when the other classifiers use a test set. That comparison would be optimistic and unfair.

14 Specialised Response Models

14.1 Unordered multiclass response

```
fit_multi <- nnet::multinom(
  class ~ x1 + x2, data = train,
  trace = FALSE
)
summary(fit_multi)
pred_multi <- predict(fit_multi,
  newdata = test)
table(test$class, pred_multi)
```

14.2 Ordered response

```
train$level <- ordered(
  train$level,
  levels = c("Low", "Medium", "High")
)

fit_ord <- MASS::polr(
  level ~ x1 + x2,
  data = train, Hess = TRUE
)
summary(fit_ord)
pred_ord <- predict(fit_ord,
  newdata = test)
```

14.3 Count response: Poisson GLM

```
fit_pois <- glm(
  count ~ x1 + x2,
  data = train,
  family = poisson(link = "log")
)
summary(fit_pois)
exp(coef(fit_pois)) # rate ratios
pred_count <- predict(
  fit_pois, newdata = test,
  type = "response"
)
```

14.4 Positive right-skewed response: Gamma

```
fit_gamma <- glm(
  y ~ x1 + x2,
  data = train,
  family = Gamma(link = "log")
)
summary(fit_gamma)
exp(coef(fit_gamma))
pred_gamma <- predict(
  fit_gamma, newdata = test,
  type = "response"
)
```

14.5 Exponential relationship via transformation

```
fit_exp <- lm(log(y) ~ x, data = train)
summary(fit_exp)

pred_log <- predict(fit_exp,
  newdata = test)
pred_y <- exp(pred_log)
```

Remember. A log link models $\log(E[Y | X]) = X\beta$. The fitted mean remains positive after applying the inverse link: $E[Y | X] = e^{X\beta}$.

15 Evaluate and Compare Models

15.1 Regression

```
rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2))
}

rmse_lm <- rmse(y_test, pred_lm)
rmse_ridge <- rmse(y_test, pred_ridge)
rmse_lasso <- rmse(y_test, pred_lasso)

c(MLR = rmse_lm,
  Ridge = rmse_ridge,
  LASSO = rmse_lasso)
```

Smaller test RMSE is better, provided models use the same test set and response scale.

15.2 Classification

```
cm <- table(Actual = actual_class,
  Predicted = pred_class)

cm
accuracy <- sum(diag(cm)) / sum(cm)
accuracy
```

For binary classes, first confirm which level is treated as the positive class before interpreting sensitivity or specificity.

16 Package Reference

Install a package only once, then load it in each new R session.

```
# Run once on your computer
install.packages(c(
  "BSDA", "car", "agricolae", "lmtest",
  "caret", "leaps", "glmnet", "e1071",
  "nnet", "ResourceSelection", "ISLR"
))

# Load when needed
library(MASS)
library(caret)
library(glmnet)
```

Package	Functions used
BSDA	z.test()
car	leveneTest(), vif()
agricolae	LSD.test()
lmtest	dwtest()
caret	createDataPartition()
leaps	regsubsets()
glmnet	glmnet(), cv.glmnet()
MASS	lda(), qda(), polr()
e1071	naiveBayes()
nnet	multinom()
ResourceSelection	hoslem.test()

17 Common Errors and Quick Fixes

Message or problem	What to check
object not found	Spelling, capitalisation and execution order
could not find function	Install/load the required package or use <code>package::function()</code>
unexpected symbol	Missing comma, quote, bracket or parenthesis
replacement has ... rows	Assigned vector length differs from data rows
contrasts can be applied...	A factor has only one observed level in the fitted data
NA/NaN/Inf	Missing values, division by zero or invalid transformation
Prediction error	Train/test columns or factor levels do not match
Matrix error in glmnet	Convert predictors with <code>model.matrix()</code> and remove response

```
# Useful checks when code fails
names(dat)
str(dat)
class(dat$y)
levels(dat$group)
dim(train); dim(test)
head(train)
traceback() # inspect latest error path
```

18 Reusable End-to-End Skeleton

```
# 1. Import and inspect
dat <- read.csv("data.csv")
head(dat); str(dat); summary(dat)
colSums(is.na(dat))

# 2. Clean and code categories
dat <- na.omit(dat)
dat$group <- factor(dat$group)

# 3. Explore
hist(dat$y)
plot(dat$x1, dat$y)
cor.test(dat$x1, dat$y)

# 4. Split before predictive modelling
set.seed(42)
id <- sample(seq_len(nrow(dat)),
             round(0.8 * nrow(dat)))
train <- dat[id, ]
test <- dat[-id, ]

# 5. Fit a model suited to the response
fit <- lm(y ~ ., data = train)
# Binary alternative:
# fit <- glm(y ~ ., data = train,
#           family = binomial)

# 6. Inspect and diagnose
summary(fit)
confint(fit)
plot(fit)

# 7. Predict unseen cases
pred <- predict(fit, newdata = test)
# For logistic probabilities, add:
# type = "response"

# 8. Evaluate
rmse <- sqrt(mean((test$y - pred)^2))
rmse

# 9. Interpret in the application
# State the model, evidence, test result,
# practical meaning and limitations.
```

19 Final Coding Checklist

1. Did I identify the response type before choosing a function?
2. Did I inspect names, types, missing values and factor levels?
3. Did I write the formula with the response on the left?
4. Did I create the train–test split before model fitting?
5. Did competing models use the same observations?
6. Did I inspect assumptions or distributional conditions?
7. Did I use `newdata = test` for final predictions?
8. Did I calculate the correct evaluation measure?
9. Did I interpret coefficients on the correct scale?
10. Can another person reproduce the analysis using my code?

Remember. R code is the implementation, not the conclusion. A complete answer explains why the method fits the response, what the output means, how the model performs on unseen data and what limitation remains.